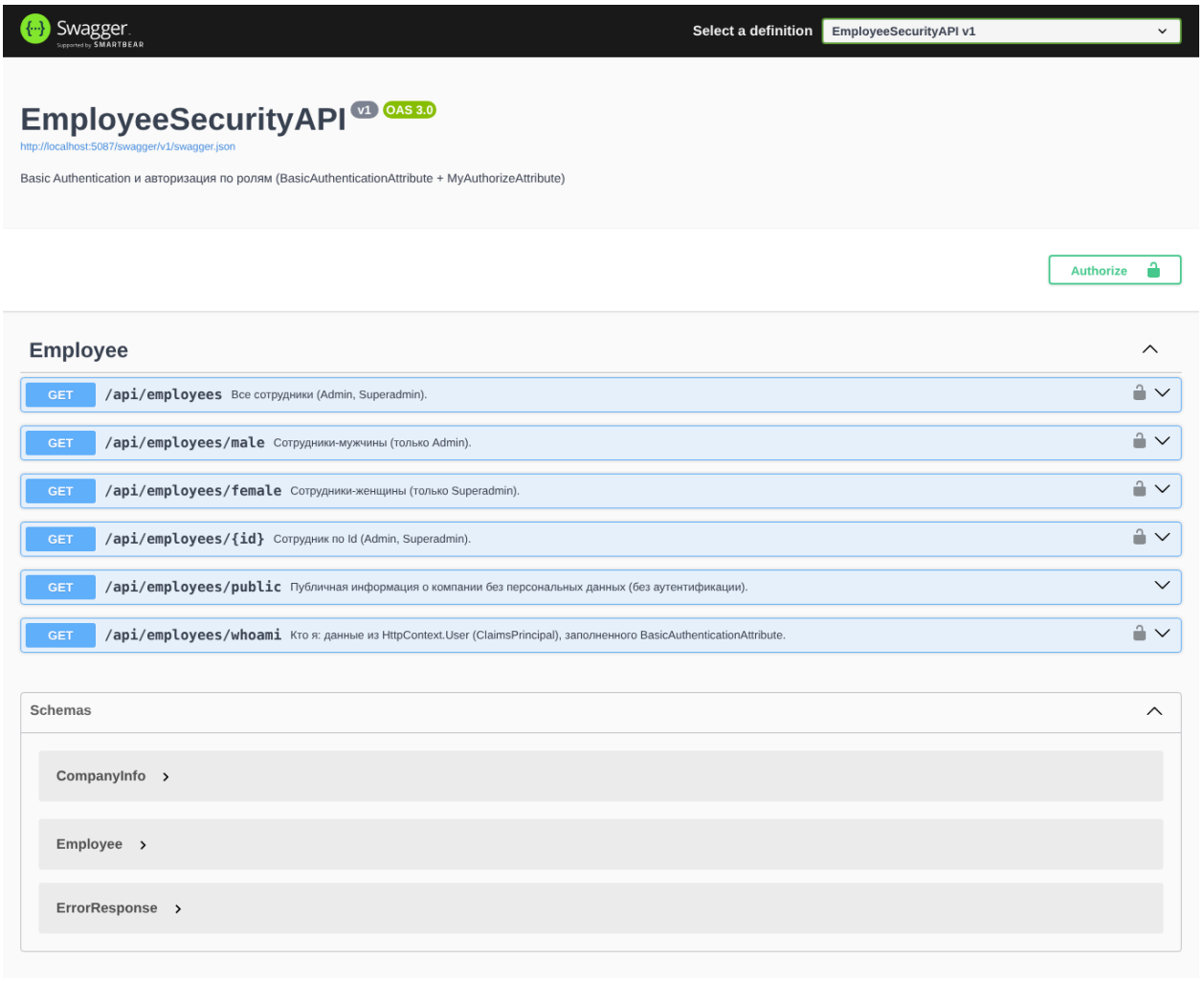
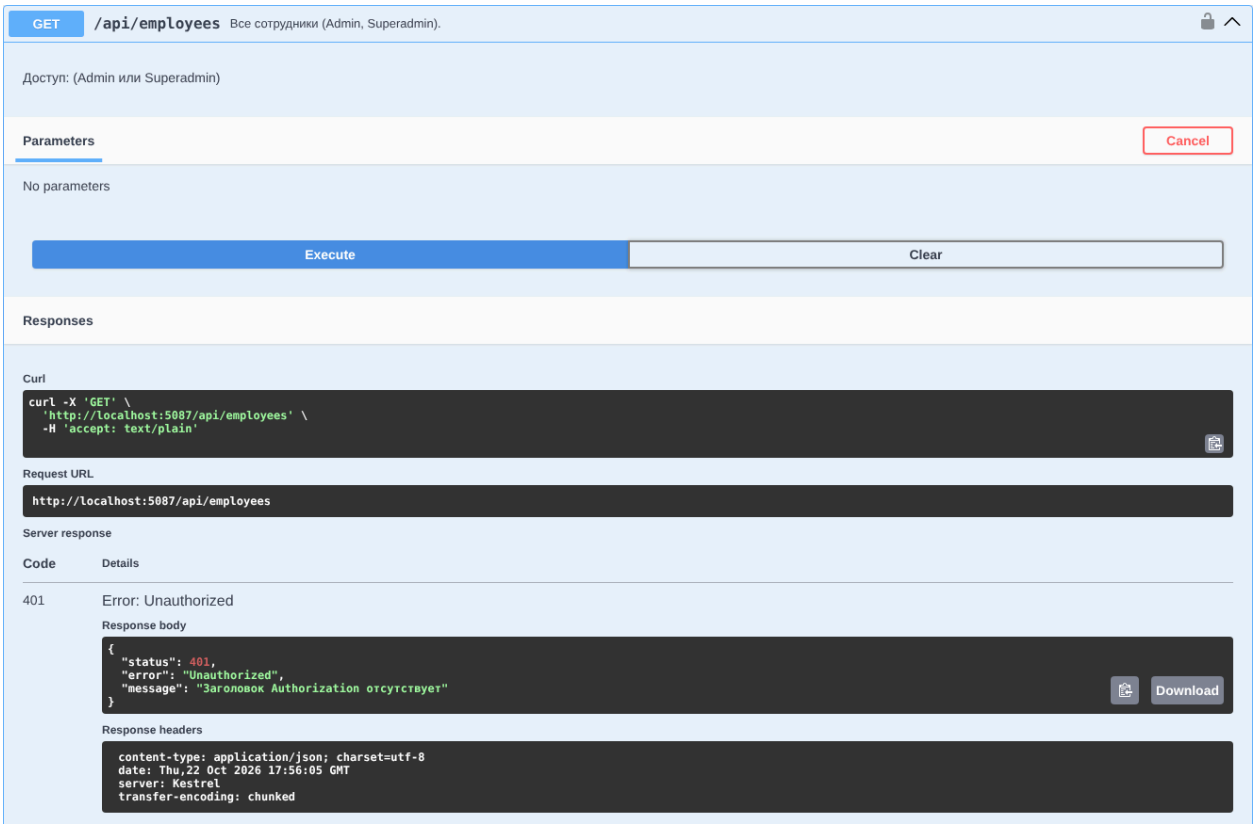
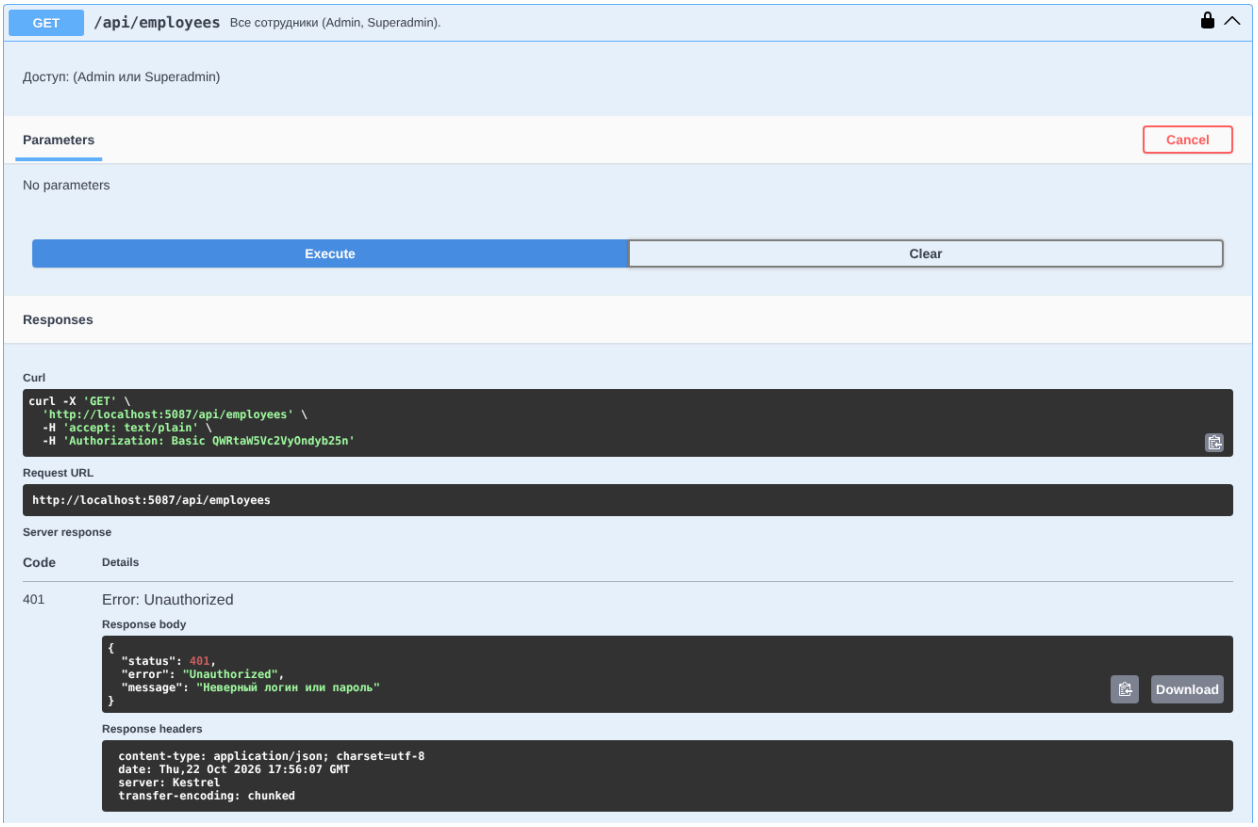
# Практическая работа 7



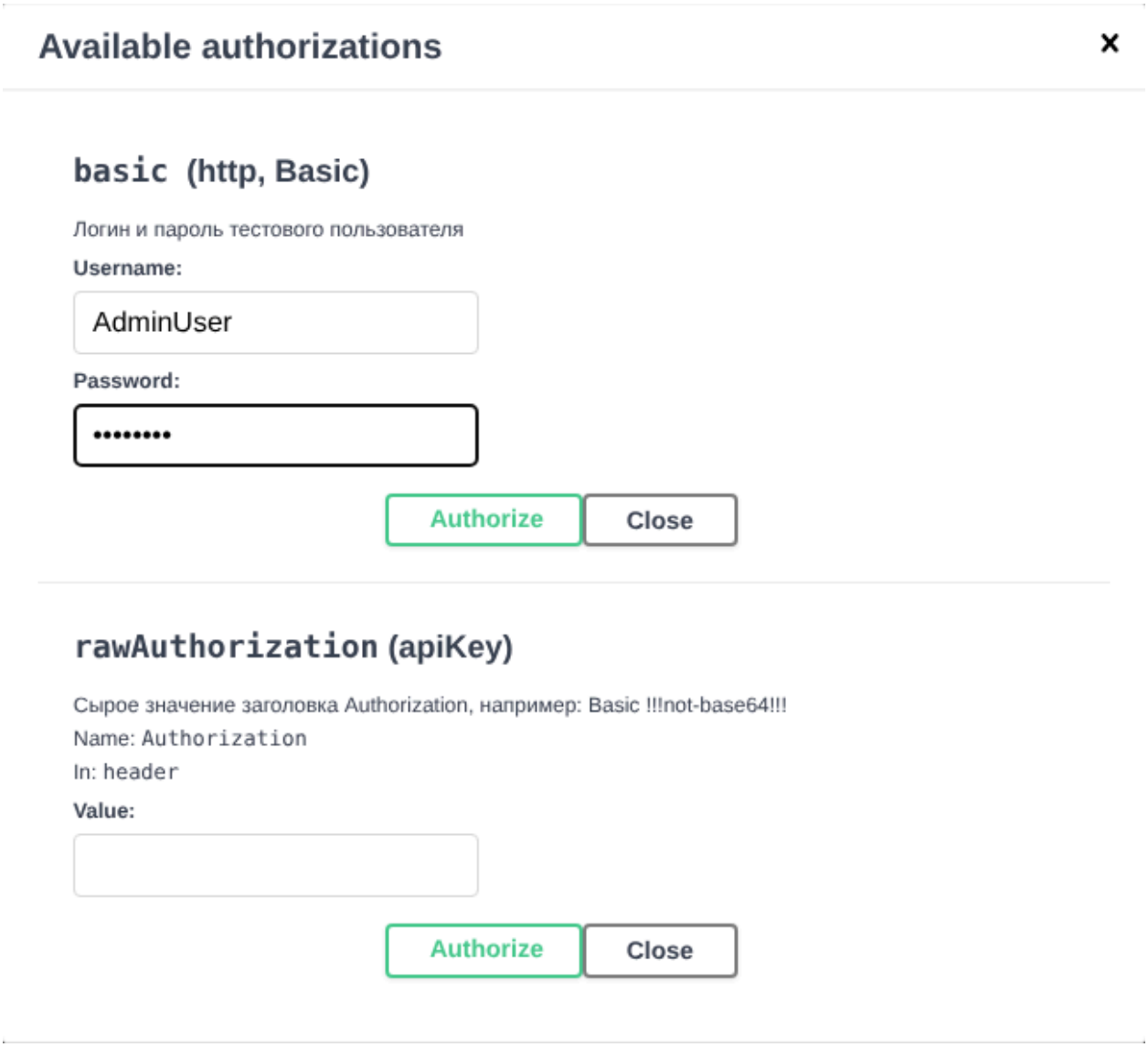

1 - Swagger UI EmployeeSecurityAPI: защищённые методы с замком, /api/employees/public



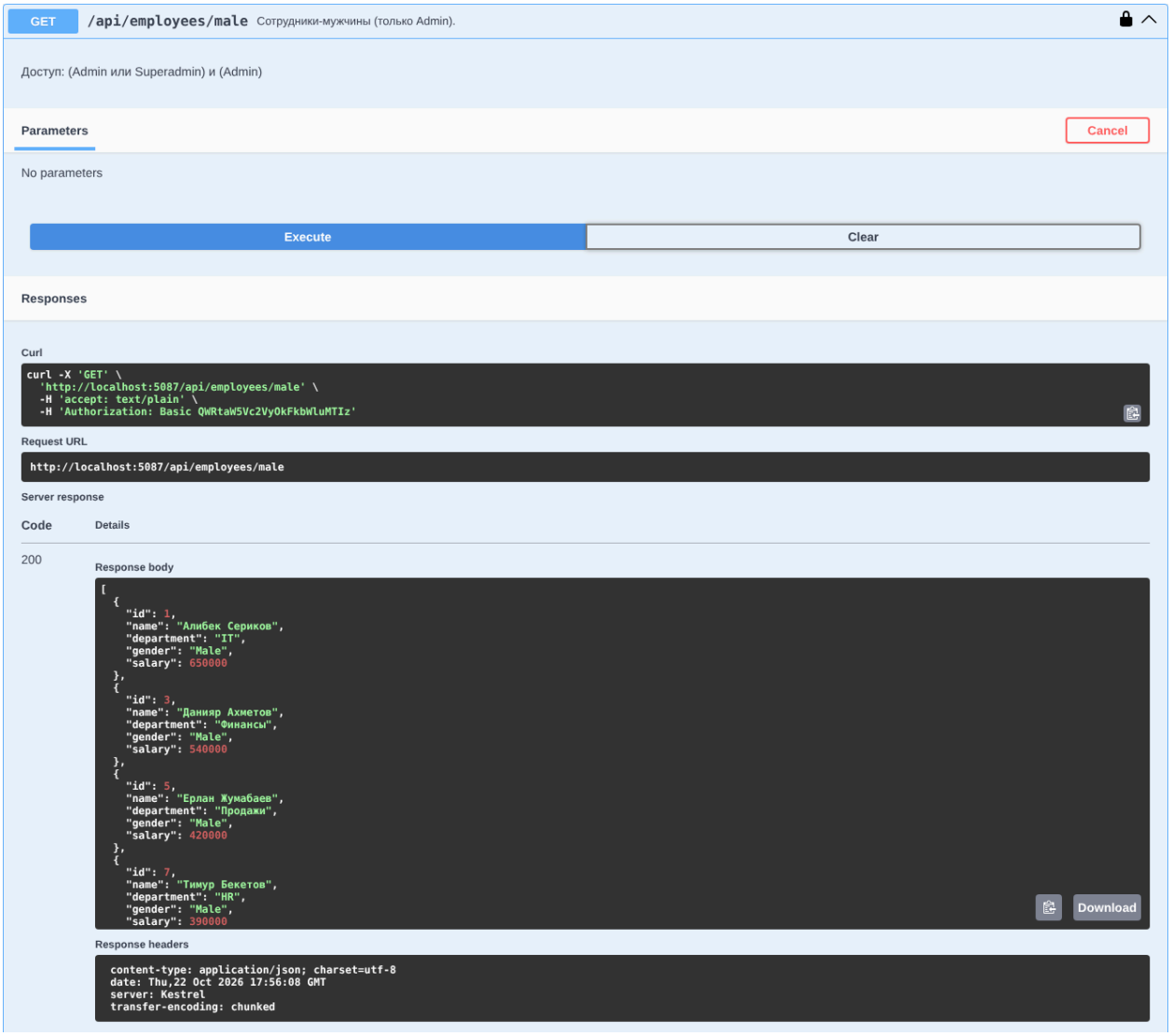

2 - Сценарий 1. Запрос без логина и пароля к GET /api/employees - 401 Unauthorized



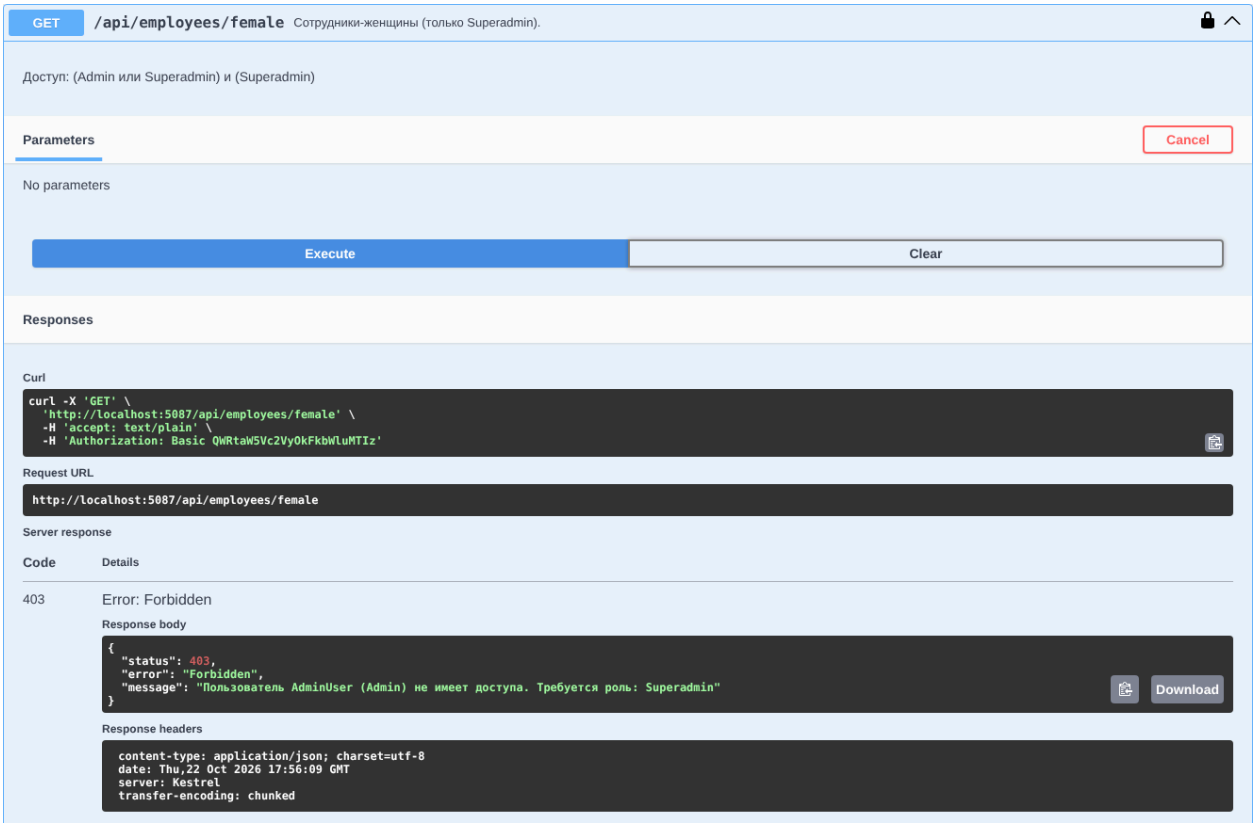

3 - Сценарий 2. Неверный пароль (AdminUser / wrong) - 401 Unauthorized



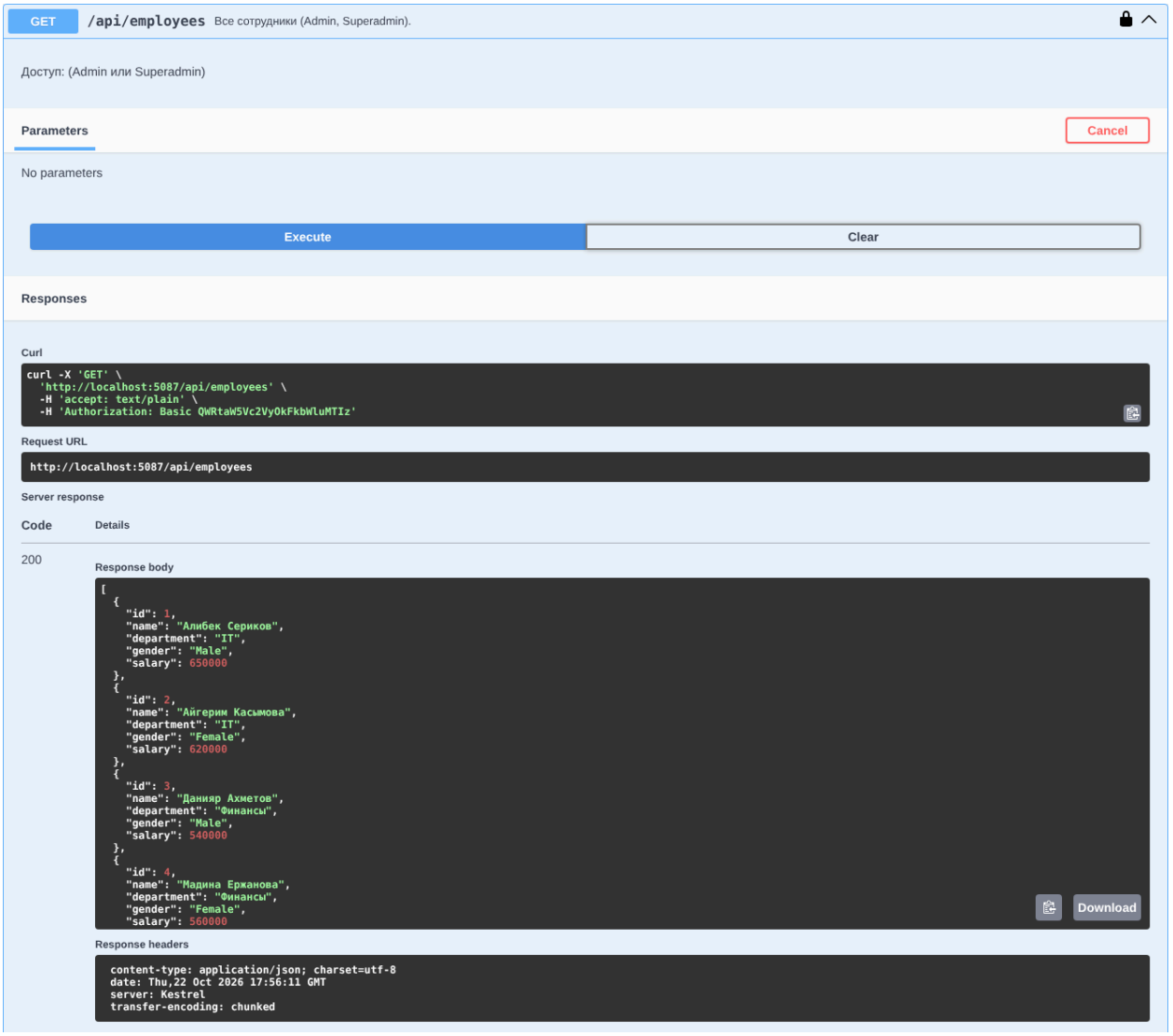

4 - Окно Authorize: Basic-аутентификация, логин AdminUser и пароль Admin123



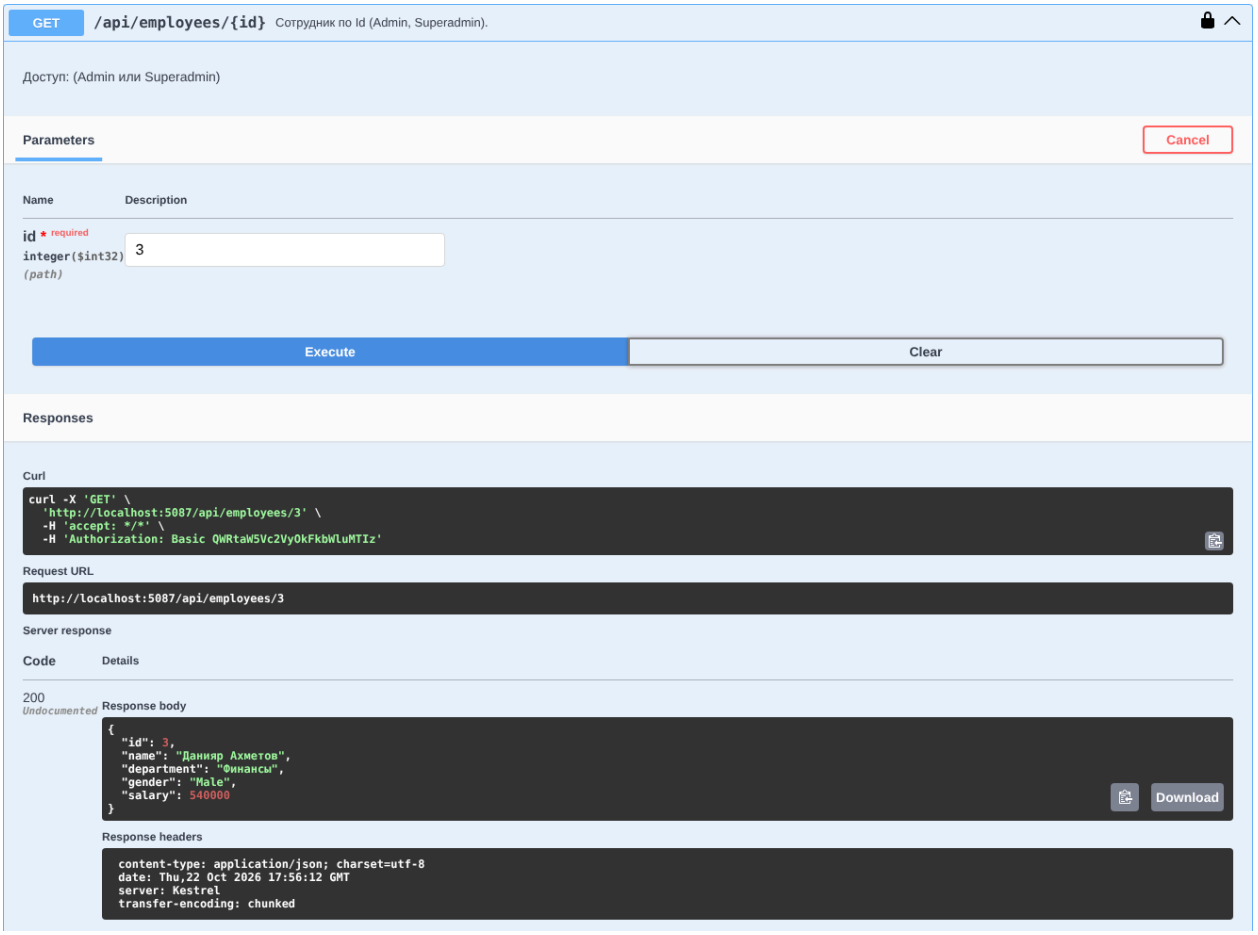

5 - Сценарий 3. Admin запрашивает /api/employees/male - 200 OK



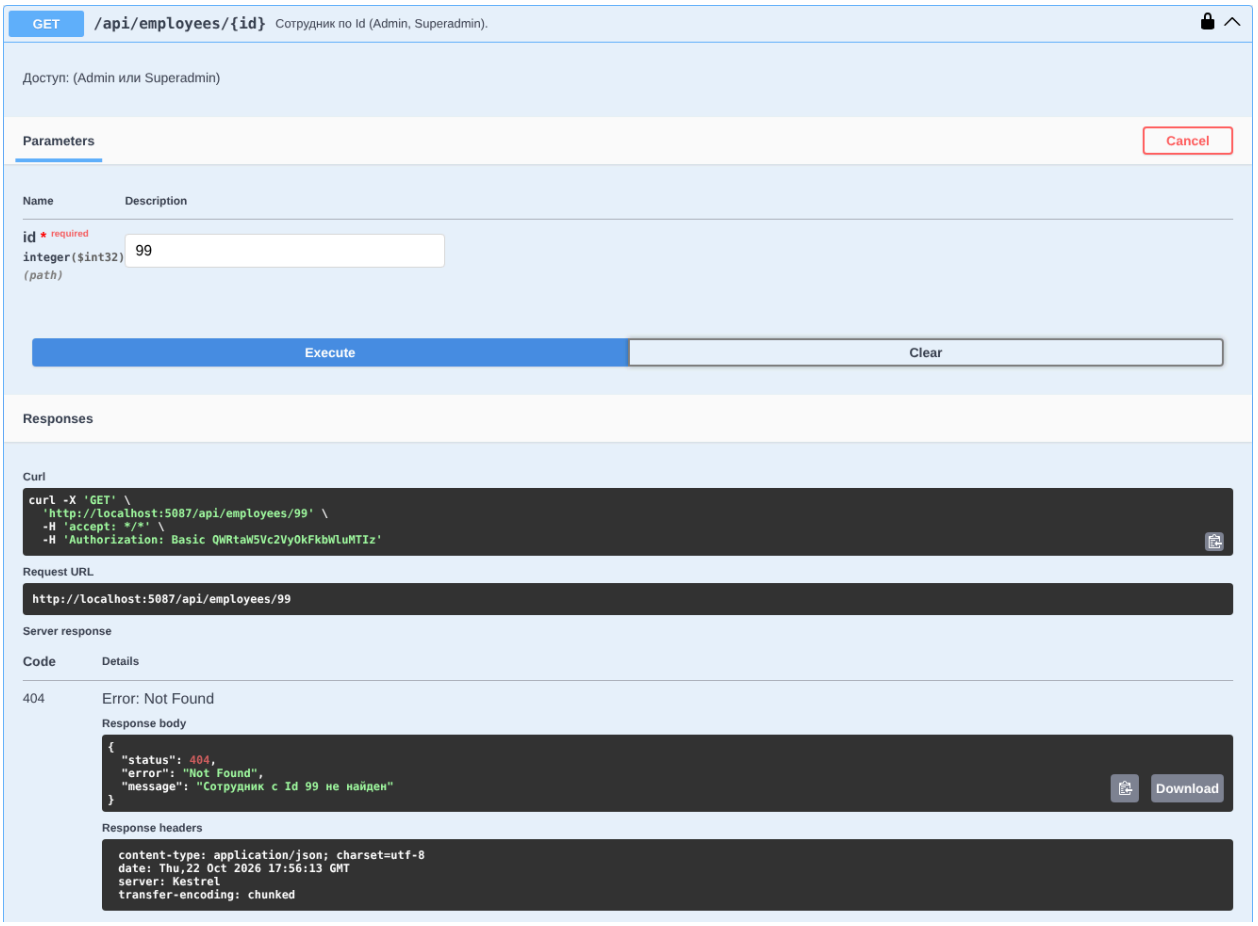

6 - Сценарий 4. Admin запрашивает /api/employees/female - 403 Forbidden



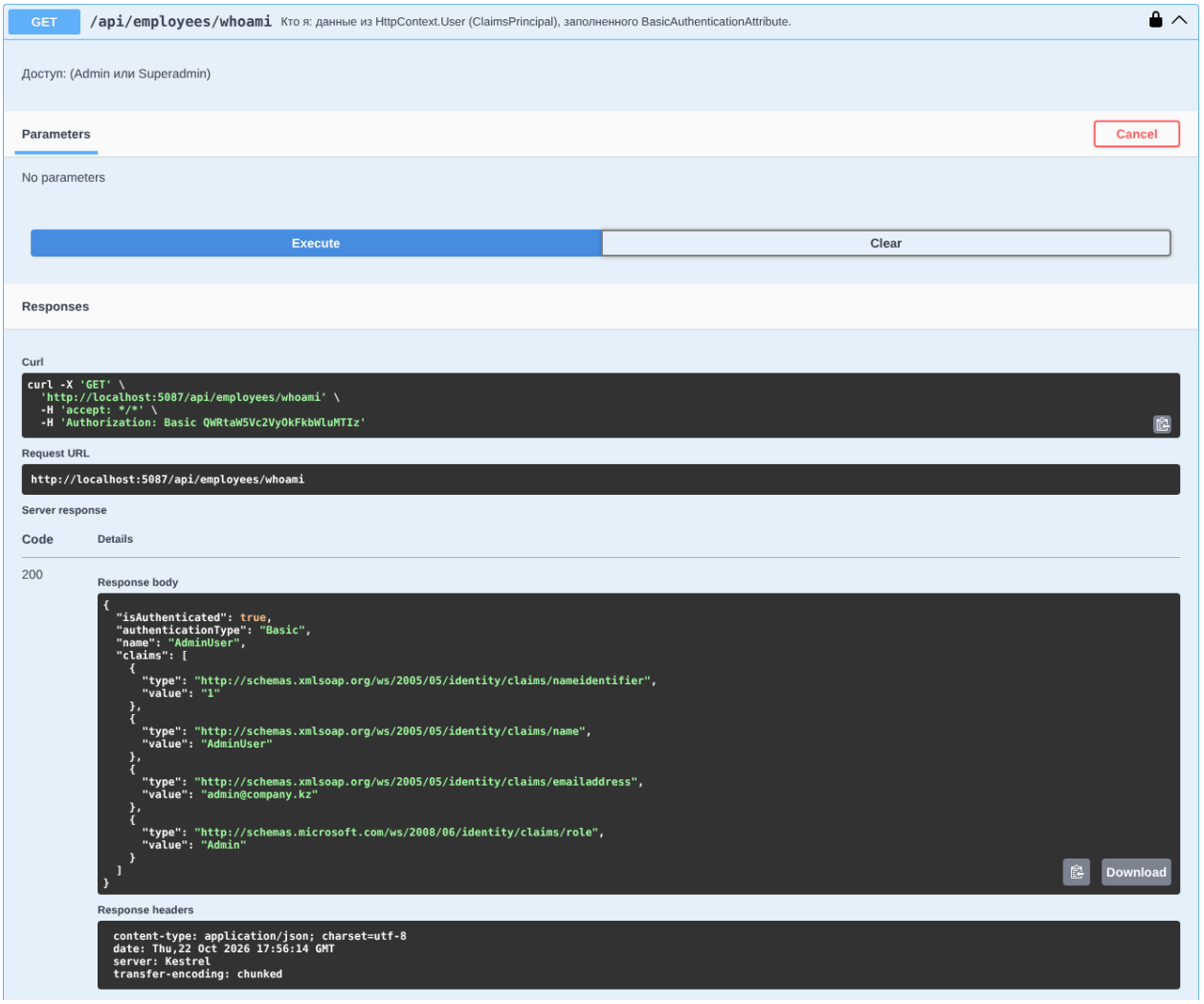

7 - AdminUser: GET /api/employees - 200 OK, 12 сотрудников



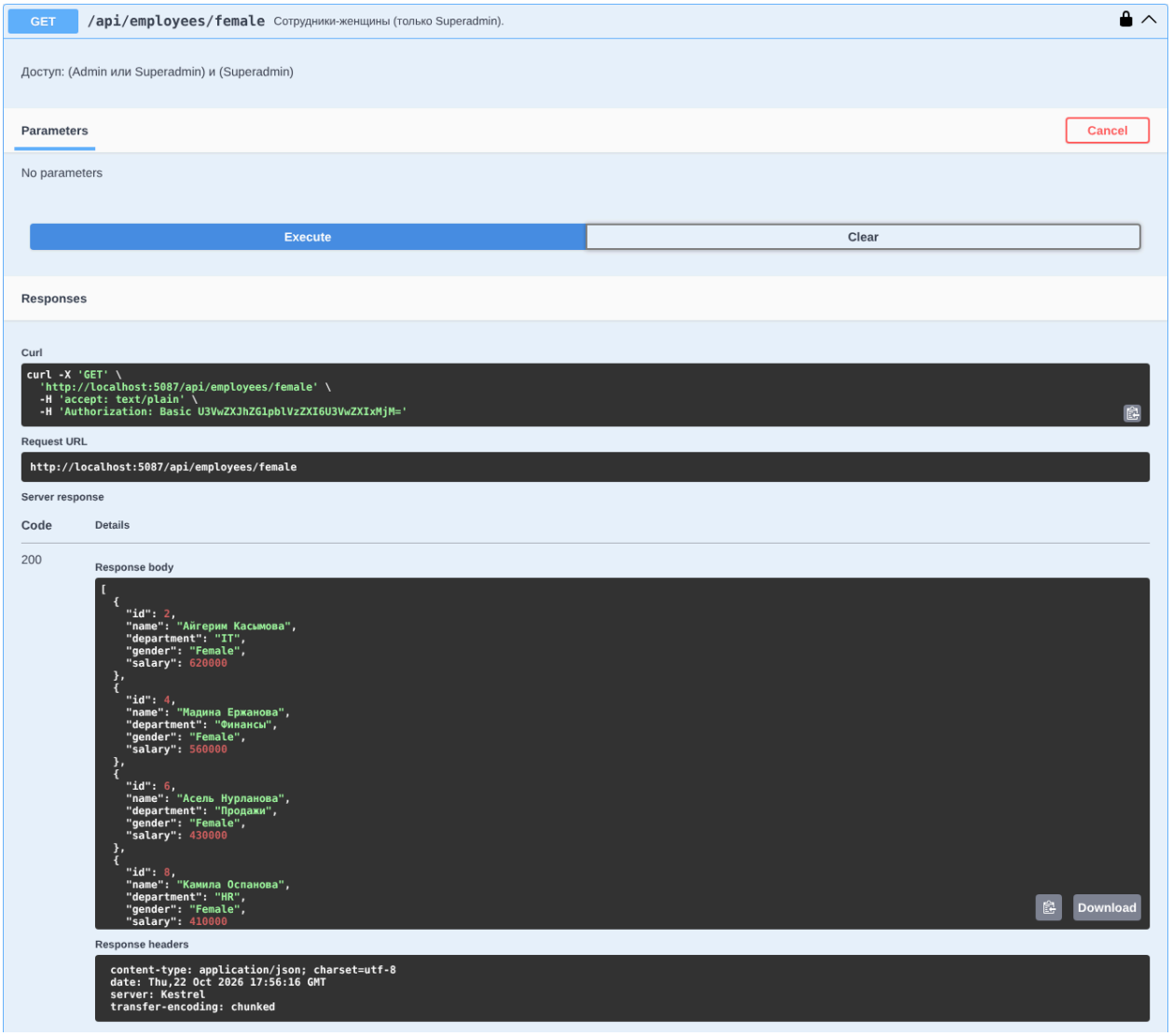

8 - AdminUser: GET /api/employees/3 - 200 OK



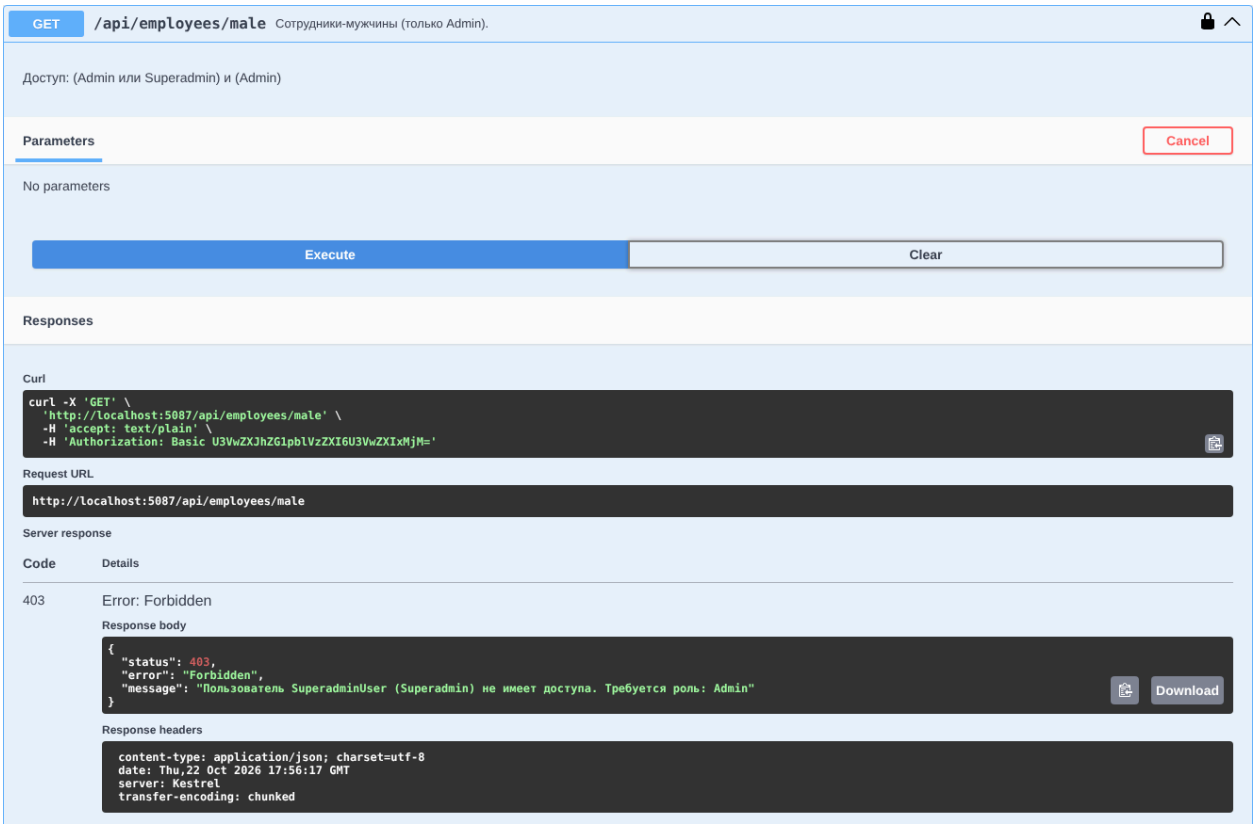

9 - AdminUser: GET /api/employees/99 - 404 Not Found



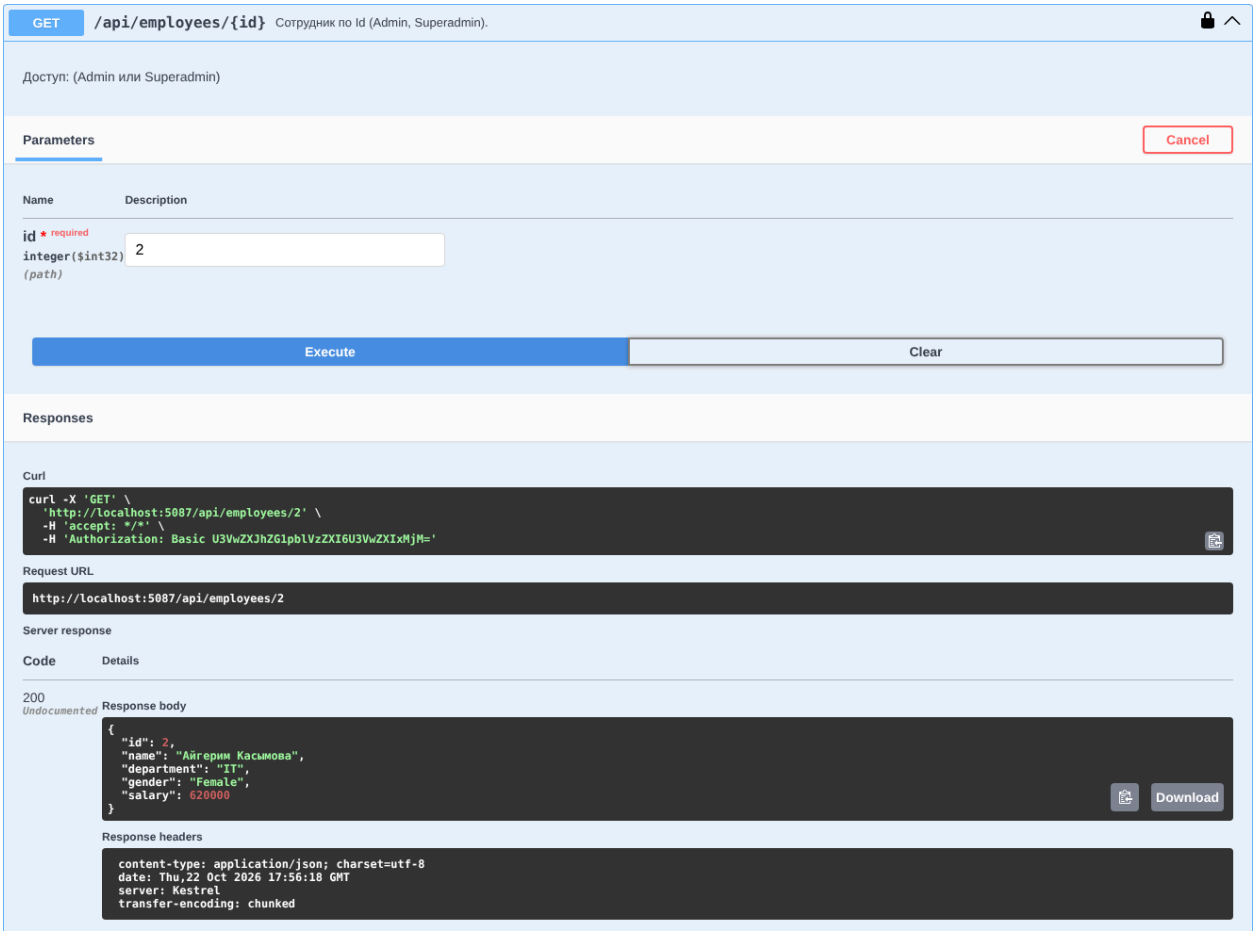

10 - AdminUser: GET /api/employees/whoami - ClaimsIdentity (Basic): Id, Name, Email, Role



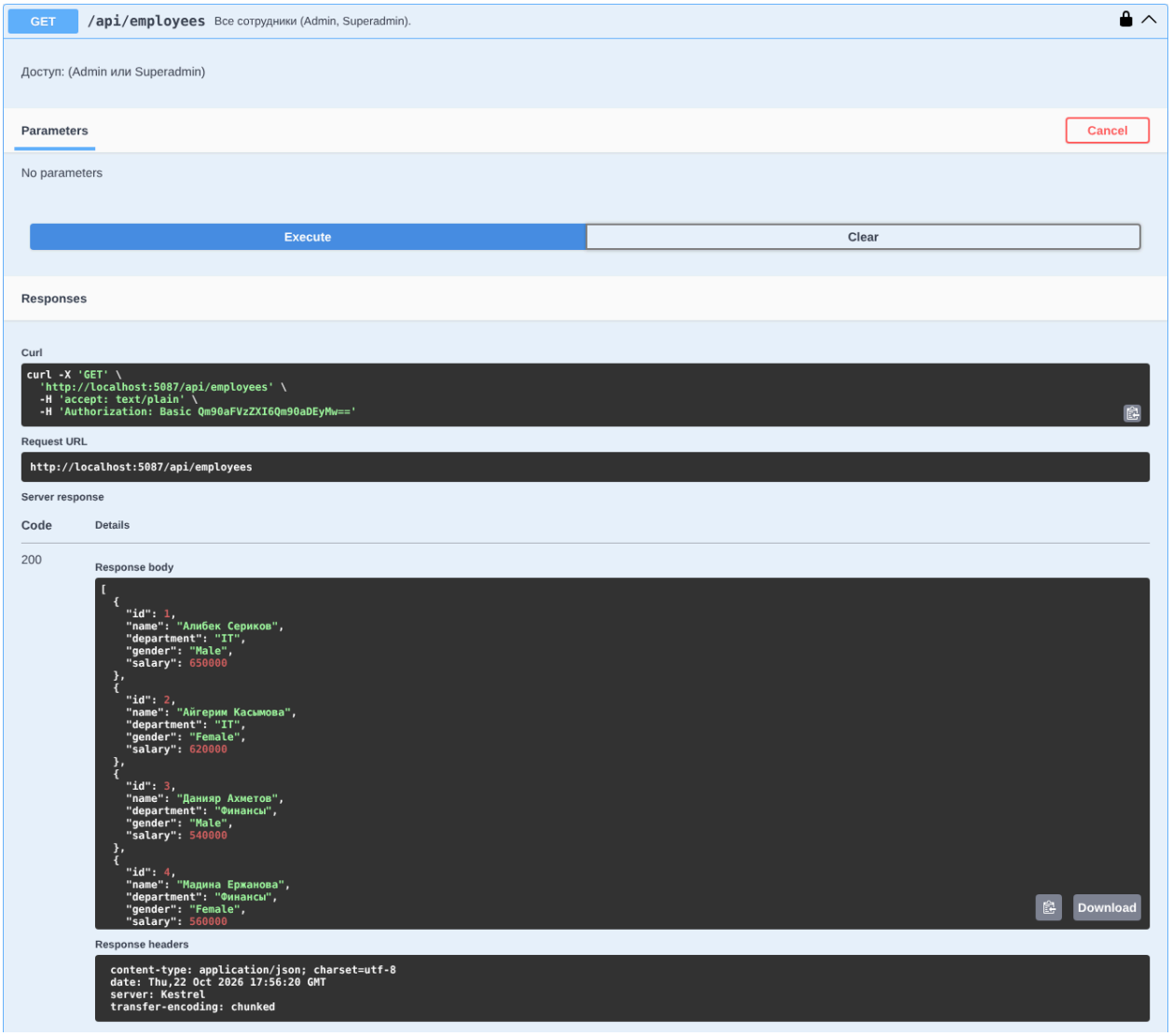

11 - Сценарий 5. Superadmin запрашивает /api/employees/female - 200 OK



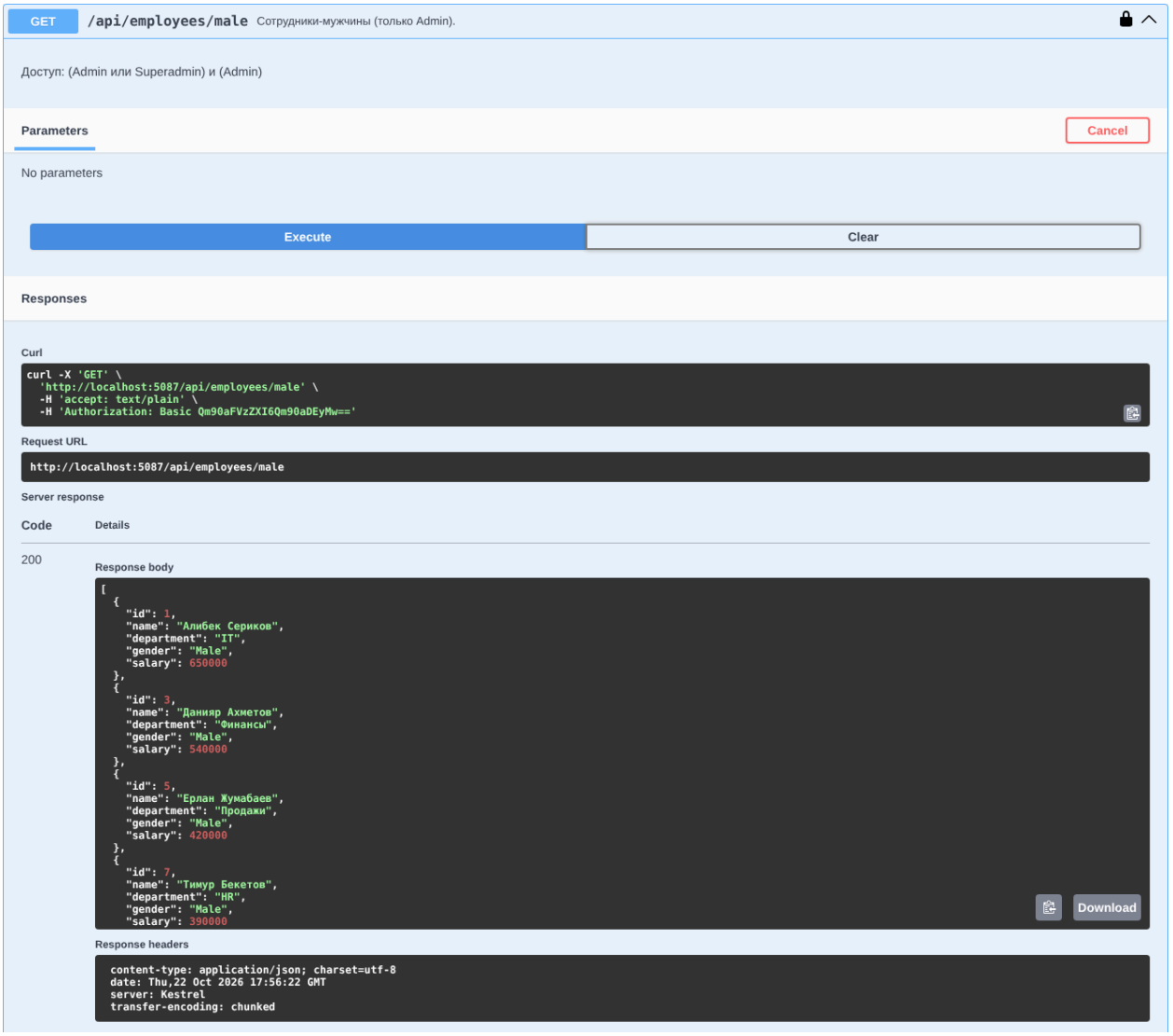

12 - Superadmin запрашивает /api/employees/male - 403 Forbidden



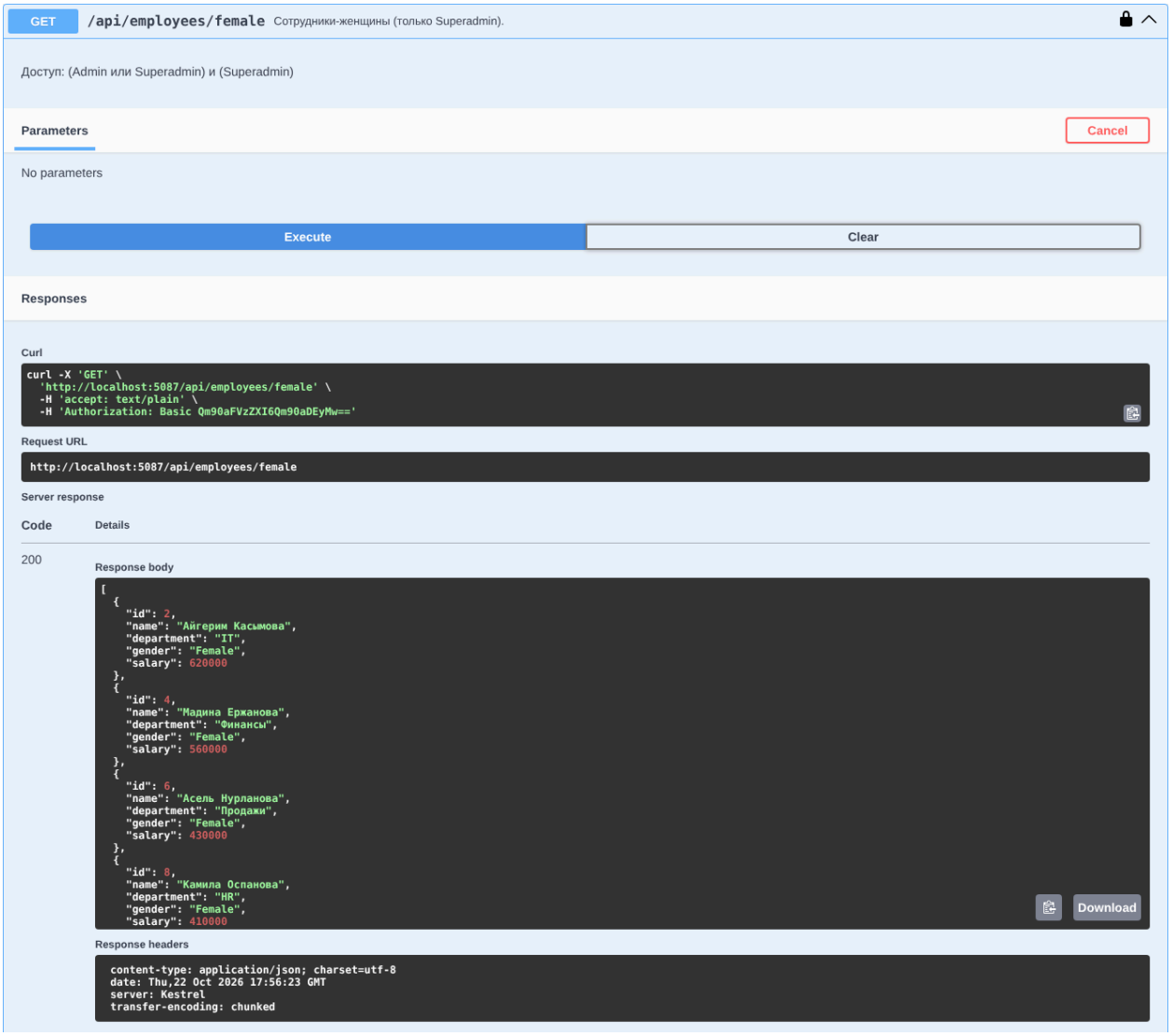

13 - SuperadminUser: GET /api/employees/2 - 200 OK



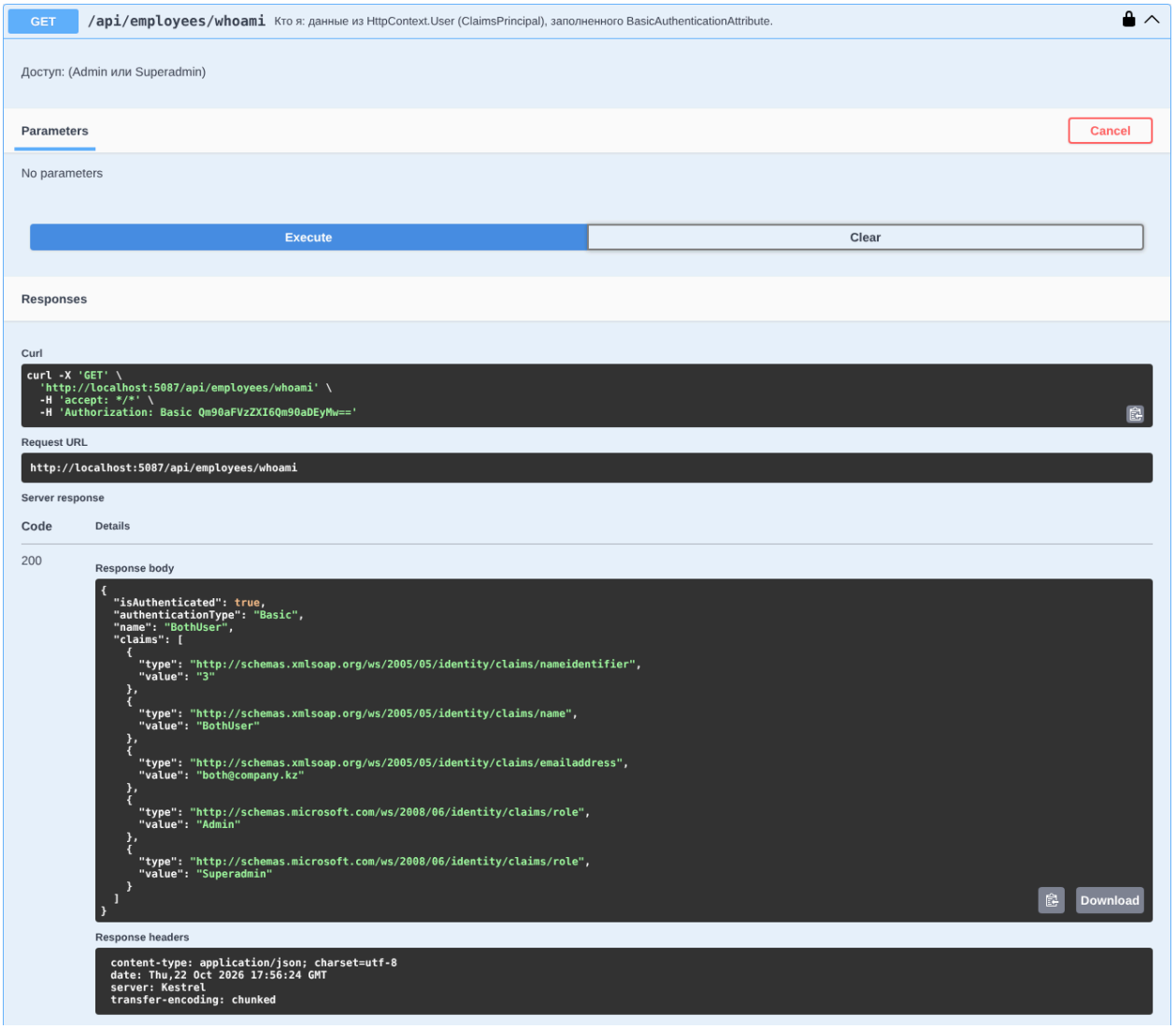

14 - Сценарий 6. BothUser (Admin, Superadmin) запрашивает /api/employees - 200 OK



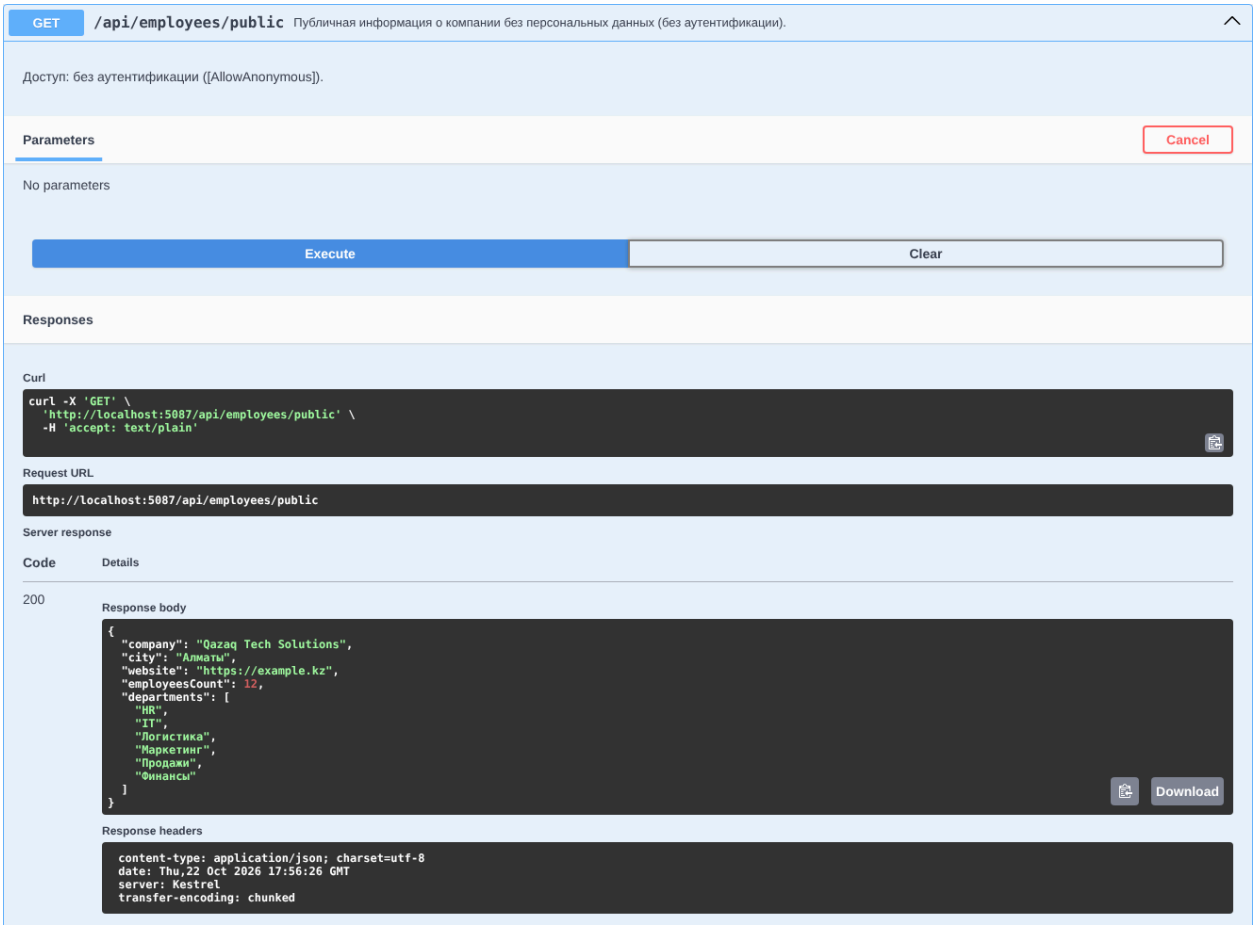

15 - BothUser: /api/employees/male - 200 OK (есть роль Admin)



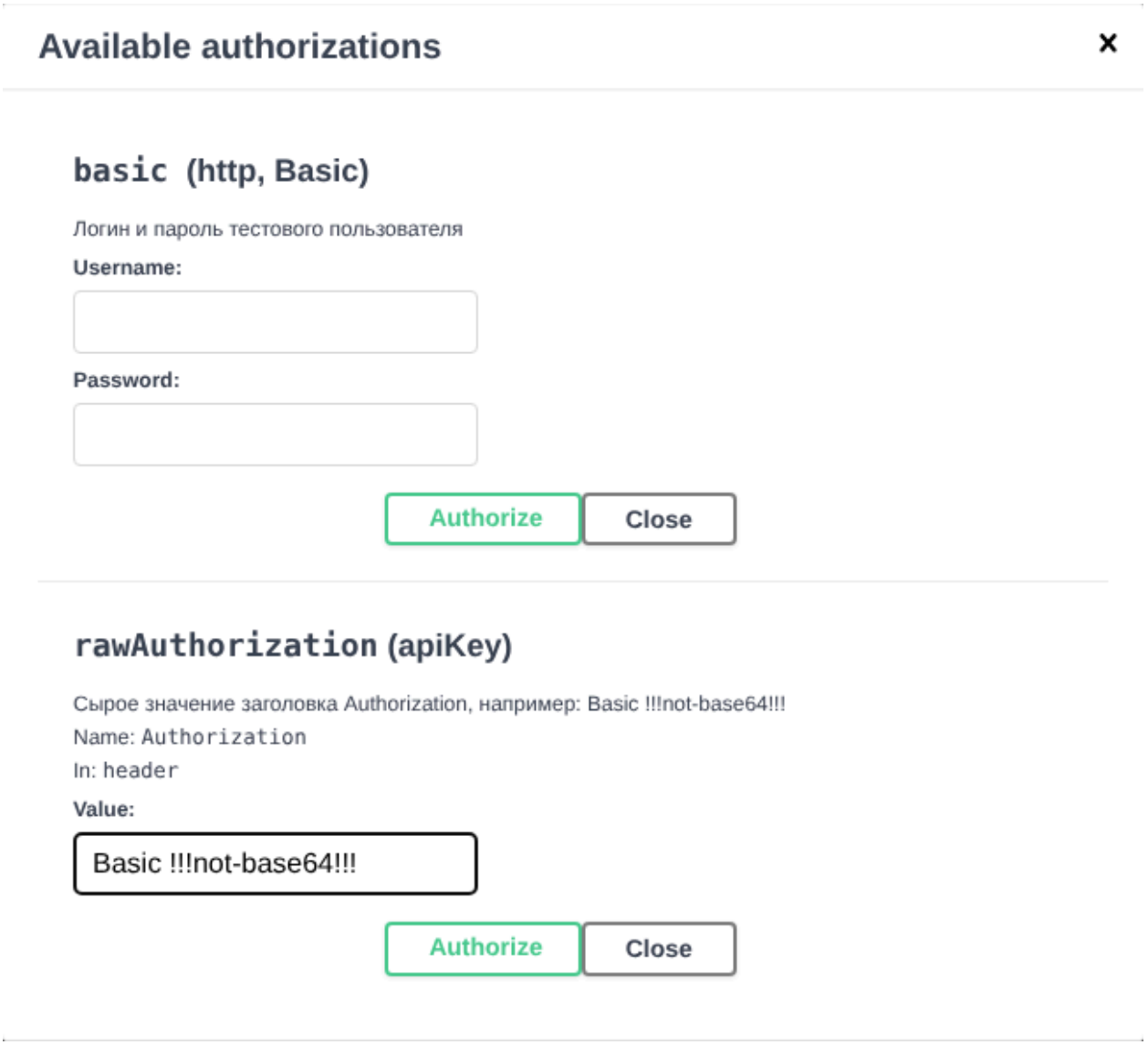

16 - BothUser: /api/employees/female - 200 OK (есть роль Superadmin)



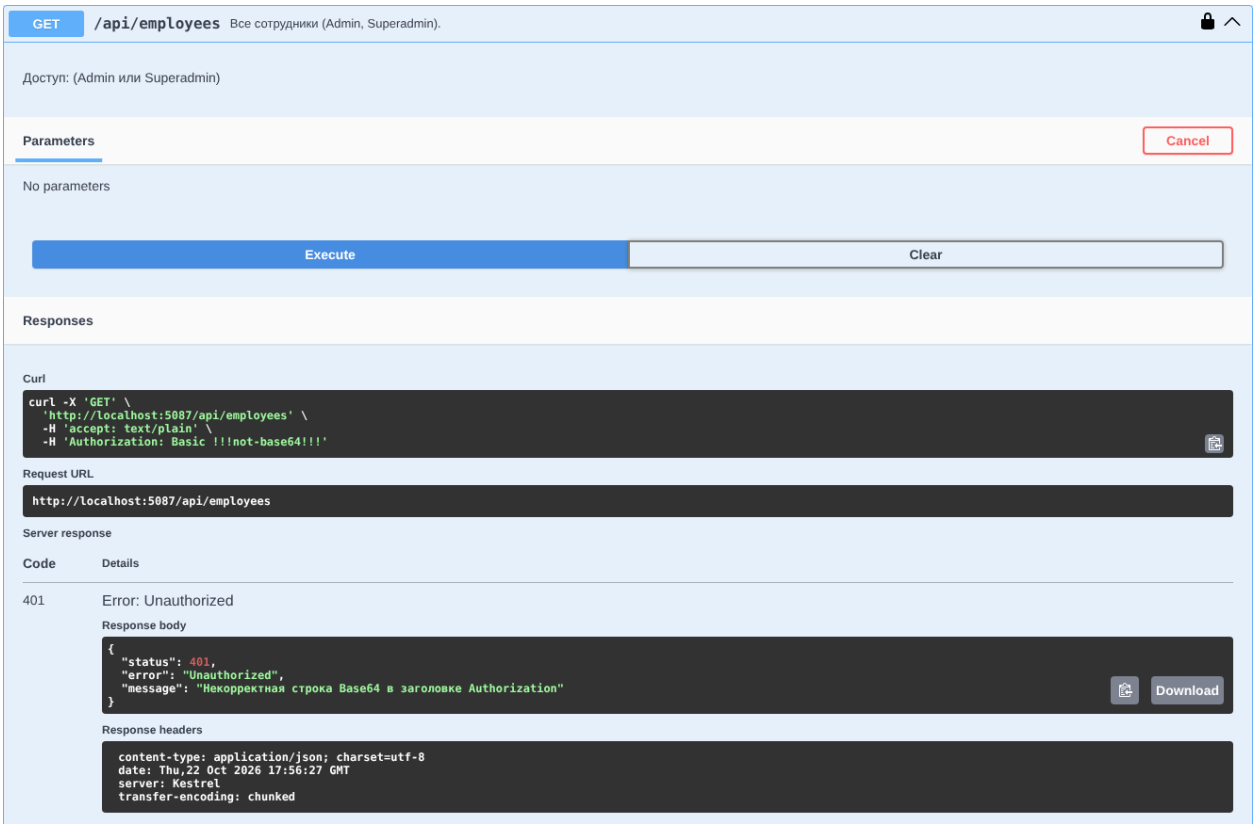

17 - BothUser: whoami - две роли в Claims



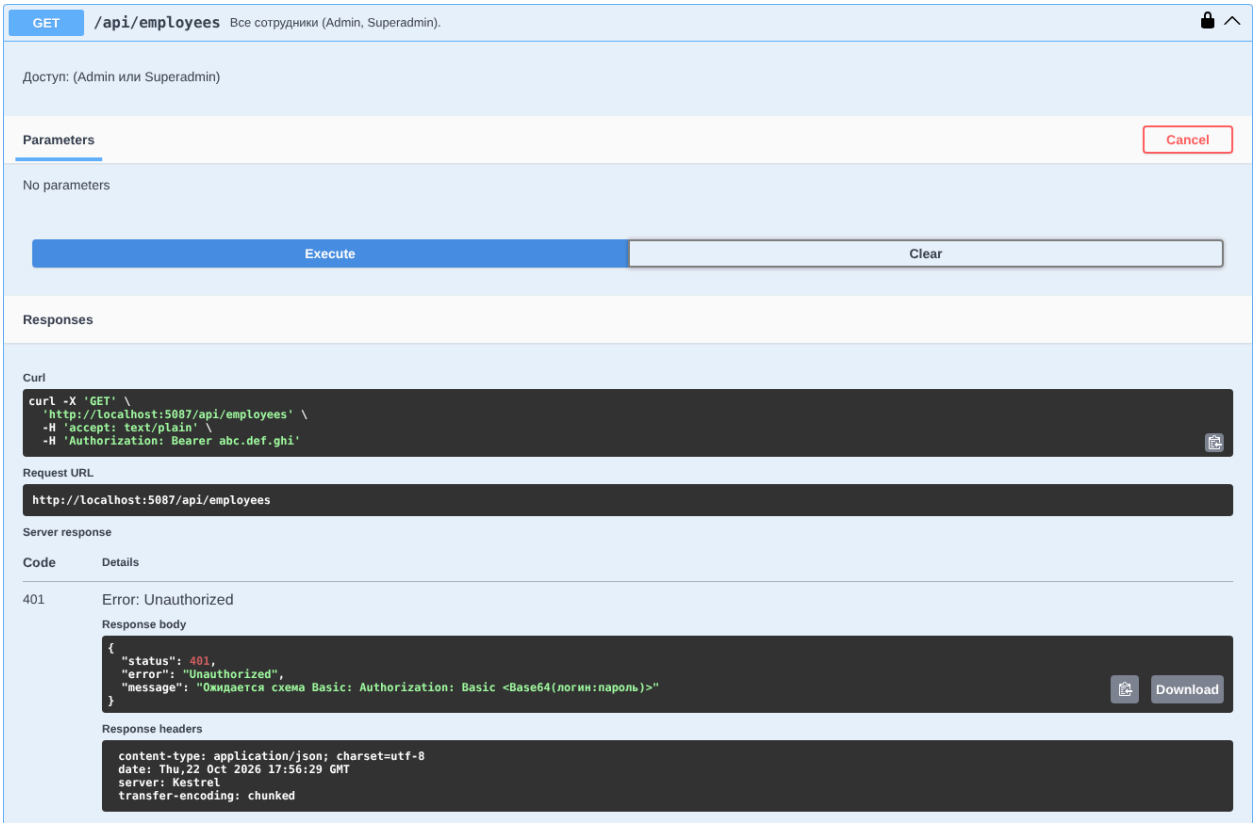

18 - Сценарий 7. Анонимный запрос к /api/employees/public - 200 OK, без персональных данных




19 - Окно Authorize: сырой заголовок Authorization с некорректной строкой Base64




20 - Сценарий 8. Некорректная строка Base64 - 401 Unauthorized




21 - Другая схема (Bearer вместо Basic) - 401 Unauthorized

## Контрольные вопросы

### 1. Чем аутентификация отличается от авторизации?

Аутентификация отвечает на вопрос «кто это?» - проверка логина и пароля в `BasicAuthenticationAttribute`; при неудаче 401. Авторизация отвечает на вопрос «что ему можно?» - проверка роли в `MyAuthorizeAttribute`; при неудаче 403. Авторизация всегда выполняется после аутентификации: AdminUser с правильным паролем аутентифицирован, но к `/female` не авторизован (рисунок 6).

### 2. Для чего используется HTTP-заголовок Authorization?

Для передачи учётных данных клиента серверу в формате `Authorization: <схема> <данные>`: `Basic QWRtaW5Vc2VyOkFkbWluMTIz`, `Bearer <JWT>` и др. Схема говорит серверу, как разобрать данные; запрос с другой схемой отклоняется (рисунок 21).

### 3. Почему Base64 не является шифрованием?

Base64 - обратимое кодирование двоичных данных в 64 печатных символа без ключа. Строку `QWRtaW5Vc2VyOkFkbWluMTIz` любой раскодирует в `AdminUser:Admin123` одной командой. Base64 нужен только для того, чтобы передать любые символы в HTTP-заголовке; конфиденциальность даёт HTTPS.

### 4. Какую роль выполняет интерфейс IPrincipal?

`IPrincipal` описывает пользователя, от имени которого выполняется код: свойство `Identity` (кто он) и метод `IsInRole(role)` (какие у него роли). ASP.NET Core хранит его в `HttpContext.User`, а фильтры авторизации и контроллеры принимают решения по нему.

### 5. Для чего используются ClaimsIdentity и ClaimsPrincipal?

`ClaimsIdentity` - одно удостоверение: набор Claims (Id, имя, Email, роли) и тип аутентификации (`"Basic"`). `ClaimsPrincipal` - пользователь, содержащий одно или несколько удостоверений; именно его записывают в `HttpContext.User`. Содержимое видно в методе `whoami` (рисунки 10, 17).

### 6. Как работает метод IsInRole()?

`ClaimsPrincipal.IsInRole("Admin")` перебирает удостоверения и ищет Claim с типом `RoleClaimType` (по умолчанию `ClaimTypes.Role`) и значением «Admin». У BothUser два таких Claim, поэтому `IsInRole` возвращает true и для Admin, и для Superadmin (рисунки 15, 16).

### 7. В чём различие между HTTP-кодами 401 и 403?

401 Unauthorized - личность не подтверждена: нет заголовка, неверный пароль, некорректный Base64; ответ содержит `WWW-Authenticate`, повтор с правильными данными может помочь. 403 Forbidden - личность подтверждена, но прав нет; повтор с теми же данными бесполезен.

### 8. Для чего используется интерфейс IAuthorizationFilter?

Это фильтр MVC, метод `OnAuthorization` которого выполняется первым в конвейере фильтров, до привязки модели и метода контроллера. Если он установит `context.Result`,

запрос завершается этим ответом. Поэтому через него удобно реализовать и аутентификацию, и проверку ролей.

**9. Чем отличается авторизация на уровне контроллера от авторизации на уровне отдельного метода?**

Атрибут на контроллере действует на все его методы, атрибут на методе - только на этот метод. Фильтры обоих уровней выполняются вместе, поэтому требования складываются: на контроллере `[MyAuthorize("Admin", "Superadmin")]`, на `/male` ещё `[MyAuthorize("Admin")]` - Superadmin проходит первый, но не второй (рисунок 12). `[AllowAnonymous]` на методе отменяет защиту контроллера (рисунок 18).

**10. Почему при использовании Basic Authentication необходимо применять HTTPS?**

Логин и пароль передаются в каждом запросе лишь в кодировке Base64. В открытом HTTP их увидит любой, кто перехватит трафик (Wi-Fi, прокси, провайдер), и сможет войти в систему. HTTPS (TLS) шифрует запрос целиком вместе с заголовком Authorization. В проекте есть профиль `https` и `UseHttpsRedirection()`.